Guion de trabajo tutorizado

MLOps: despliega tu modelo en la nube

Aplicaciones Empresariales

Profesor D. Juan Antonio Morente
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Enunciado publicado en el Aula Virtual (Campus Virtual UPO), actividad «AE 5 - MLOps: Despliega tu modelo en la nube».

La tarea a entregar consistirá en compartir en un documento (word o pdf) evidencias en forma de capturas de pantalla del proceso de despliegue del modelo de lenguaje en Google Cloud. Para ello, como se puede ver en detalle en la grabación de clase, se debe seguir el documento alojado en el zip de código, en la carpeta «docs», cualquiera de los dos llamados «gcp_deployment», en función del sistema operativo.

Las capturas de pantalla que debemos incluir son:

- Creación del bucket.
- Creación del Artifact Registry.
- Creación del Service Account.
- Creación del ingestion job y su ejecución exitosa.
- Listado de archivos de los pesos del LLM visibles en el bucket.
- Vista interior del Artifact Registry, repositorio llm-repo, con los dos contenedores (el del job y el del servidor).
- Vista del terminal habiendo ejecutado la instrucción de eliminar todos los recursos Cloud.

RESOLUCIÓN

Entorno de trabajo y parámetros del despliegue

El despliegue sigue la guía `docs/gcp_deployment.md` del código de clase. Se ha trabajado desde **Cloud Shell**, que ya incluye `gcloud` autenticado, de modo que la fase 0 se reduce a fijar las variables de entorno del proyecto.

Variable	Valor
`PROJECT_ID`	`mbda-gvdm-2026`
`REGION`	`europe-west1` (Bélgica, región con disponibilidad de GPU NVIDIA L4)
`BUCKET_NAME`	`mbda-ae05-l4-weights-1789904647`
`MODEL_ID`	`Qwen/Qwen3.5-2B`
`SA_EMAIL`	`llm-runner-identity@mbda-gvdm-2026.iam.gserviceaccount.com`
Repositorio de imágenes	`llm-repo` (Artifact Registry, formato Docker)

La arquitectura objetivo es la de la guía: un **Cloud Run Job** (`llm-ingestion-job`) descarga los pesos del modelo desde Hugging Face y los deposita en un **bucket de Cloud Storage**; ese bucket se monta después con FUSE en `/mnt/models` dentro de un **servicio de Cloud Run con GPU L4** que ejecuta **vLLM** y expone una API compatible con OpenAI. Separar la ingesta del servicio es la clave del patrón: los pesos se descargan **una sola vez** y quedan en el lago, de modo que el arranque en frío del servidor no depende de Hugging Face ni de su ancho de banda.

Fase 1. Creación del bucket de pesos

```
gcloud storage buckets create gs://$BUCKET_NAME --project=$PROJECT_ID \
  --location=$REGION --uniform-bucket-level-access
```

Se usa **acceso uniforme a nivel de bucket** (sin ACL por objeto), que es la práctica recomendada: los permisos se conceden solo mediante IAM, lo que evita objetos accidentalmente públicos. El bucket es regional y está en la **misma región que el servicio**, requisito para que el montaje FUSE tenga latencia baja y no genere tráfico entre regiones.

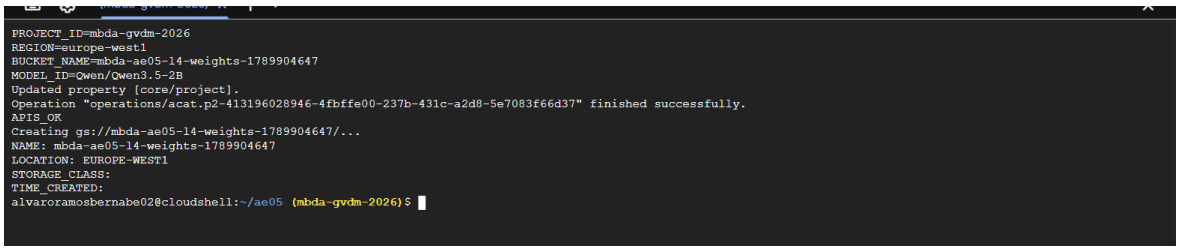


Figura 1. Creación del bucket `mbda-ae05-l4-weights-1789904647` en `europe-west1` con acceso uniforme.

Fase 2. Creación del repositorio en Artifact Registry

```
gcloud artifacts repositories create llm-repo --repository-format=docker \
  --location=$REGION --description="Repositorio de imagenes para el despliegue
del LLM"
```

Artifact Registry sustituye al antiguo Container Registry y alberga las dos imágenes del despliegue: la del *job* de ingesta y la del servidor vLLM. Al estar en la misma región que Cloud Run, el *pull* de la imagen durante el arranque es local.

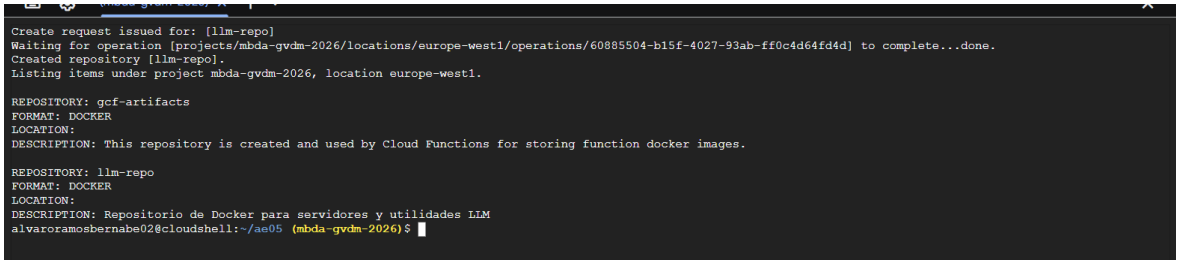


Figura 2. Creación del repositorio Docker `llm-repo` en `europe-west1`.

Fase 3. Cuenta de servicio e identidad del despliegue

En lugar de usar la cuenta de servicio por defecto de Compute Engine —que tiene permisos de editor sobre todo el proyecto—, se crea una **identidad dedicada** con los permisos mínimos necesarios, conforme al principio de mínimo privilegio:

```
gcloud iam service-accounts create llm-runner-identity \
  --display-name="Identidad para el servicio LLM y el job de ingesta"

gcloud storage buckets add-iam-policy-binding gs://$BUCKET_NAME \
  --member="serviceAccount:$SA_EMAIL" --role="roles/storage.objectAdmin"   #
escritura: job de ingesta

gcloud storage buckets add-iam-policy-binding gs://$BUCKET_NAME \
  --member="serviceAccount:$SA_EMAIL" --role="roles/storage.objectViewer"  #
lectura: montaje FUSE
```

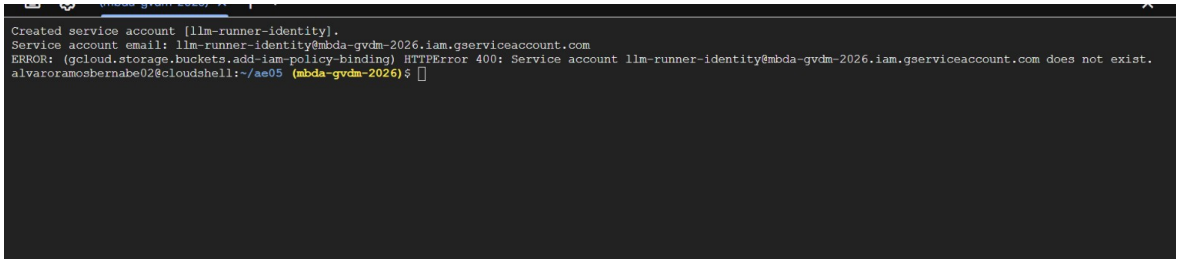


Figura 3. Creación de la cuenta de servicio `llm-runner-identity`.



Figura 4. Concesión de los roles `objectAdmin` y `objectViewer` sobre el bucket de pesos.

Los roles se conceden **sobre el bucket**, no sobre el proyecto: la identidad puede escribir y leer esos objetos y nada más. Durante la ejecución se produjo un error transitorio de propagación de IAM al añadir el segundo enlace; se resolvió reintentando la orden pasados unos segundos, un comportamiento habitual porque la política de IAM es de consistencia eventual.

Fase 4. Job de ingesta: construcción, despliegue y ejecución

El *job* (`jobs/llm-ingestion`) es un contenedor Python que lista los ficheros del repositorio `Qwen/Qwen3.5-2B` en Hugging Face, descarta los formatos innecesarios (`*.msgpack`, `*.h5`, `*.ot`) y sube el resto al bucket. Primero se construye la imagen con Cloud Build y después se despliega y ejecuta el *job*:

```
gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT_ID/llm-repo/llm-
ingestion:latest ./jobs/llm-ingestion

gcloud run jobs deploy llm-ingestion-job \
  --image=$REGION-docker.pkg.dev/$PROJECT_ID/llm-repo/llm-ingestion:latest \
  --region=$REGION --service-account=$SA_EMAIL \
  --set-env-vars="BUCKET_NAME=$BUCKET_NAME,MODEL_ID=$MODEL_ID" \
  --memory=4Gi --cpu=2 --task-timeout=3600

gcloud run jobs execute llm-ingestion-job --region=$REGION --wait
```

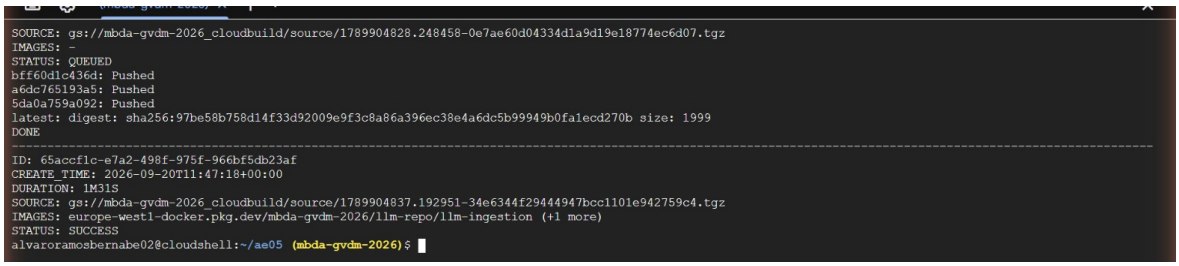


Figura 5. Cloud Build: construcción de la imagen `llm-ingestion` y publicación en `llm-repo` (estado SUCCESS).

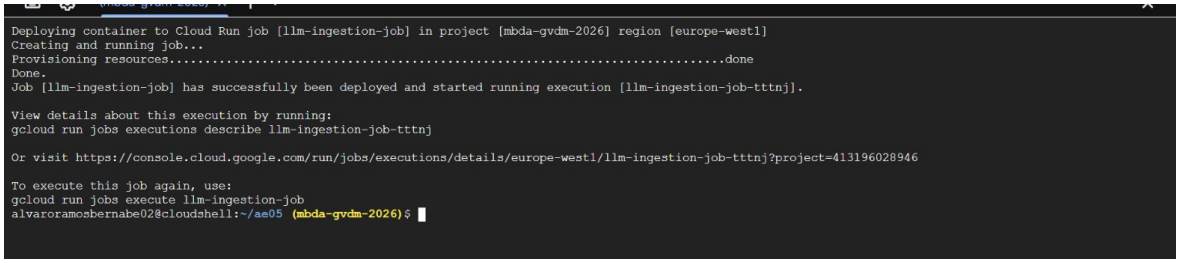


Figura 6. Despliegue del Cloud Run Job `llm-ingestion-job` con la cuenta de servicio dedicada.

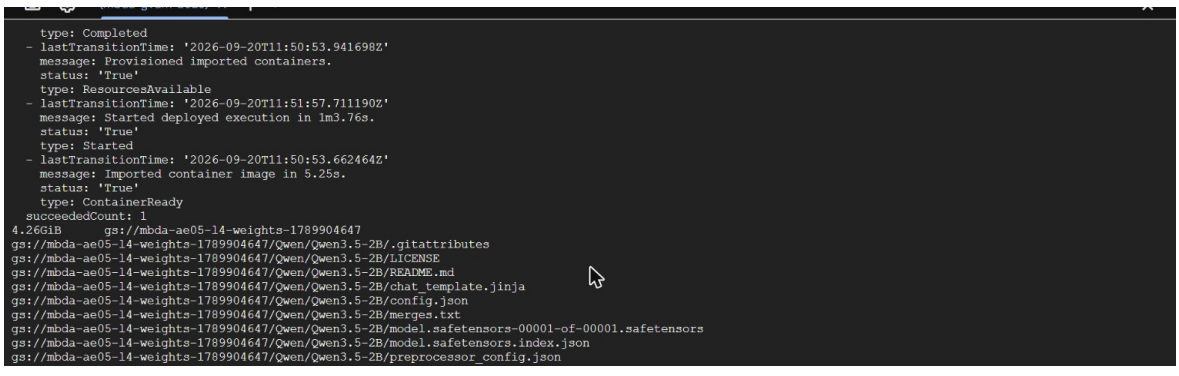


Figura 7. Ejecución `llm-ingestion-job-tttnj` finalizada con éxito desde el terminal.

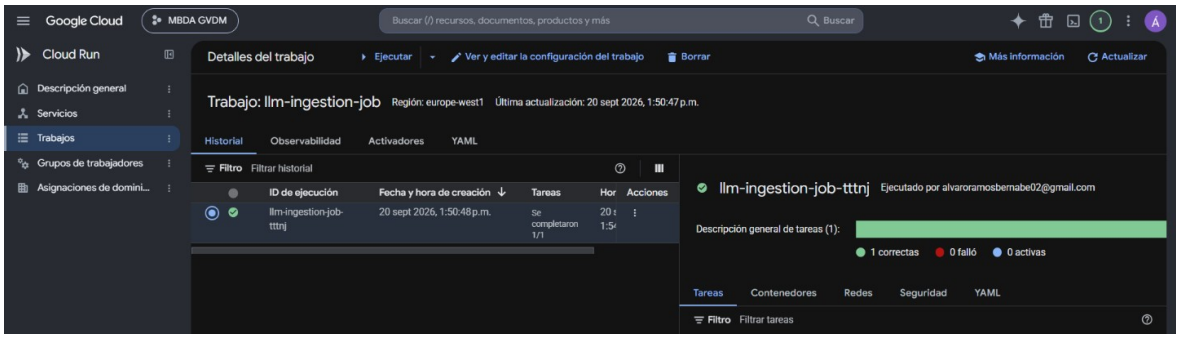


Figura 8. La misma ejecución vista en la consola de Cloud Run: 1 tarea completada.

Un *job* es la primitiva adecuada aquí —y no un servicio— porque la ingesta es una tarea **finita** que termina cuando acaba la descarga: no atiende peticiones HTTP, no necesita escalar y no debe permanecer levantada. Los parámetros de memoria y `task-timeout` se dimensionan para el tamaño del modelo.

Fase 5. Pesos del modelo en el bucket

```
gcloud storage ls -l gs://$BUCKET_NAME/$MODEL_ID/
```

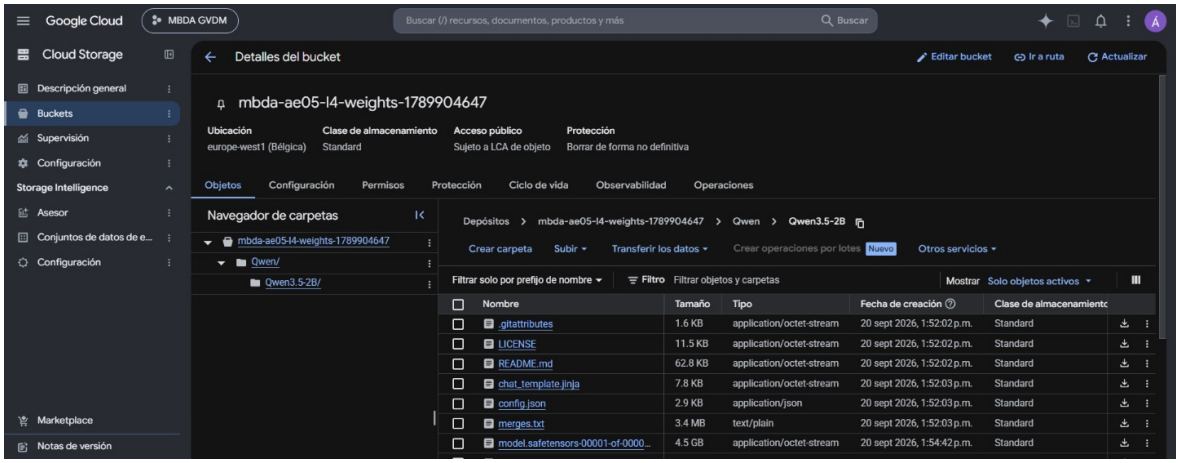


Figura 9. Listado de los ficheros del modelo en `gs://mbda-ae05-l4-weights-1789904647/Qwen/Qwen3.5-2B/`: pesos en formato safetensors, configuración y tokenizador (4,26 GiB en total).

Aparecen el fichero de pesos en formato **safetensors**, el `config.json`, el `generation_config.json` y los ficheros del tokenizador: exactamente lo que vLLM necesita encontrar en `/mnt/models` para cargar el modelo sin volver a descargarlo.

Fase 6. Imagen del servidor vLLM y manifiesto del servicio

La segunda imagen contiene el servidor de inferencia. Se construye igual que la anterior y queda publicada en el mismo repositorio:

```
gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT_ID/llm-repo/vllm-
qwen:latest ./services/llm

sed "s|__PROJECT_ID__|$PROJECT_ID|g; s|__REGION__|$REGION|g; s|__BUCKET_NAME__|
$BUCKET_NAME|g; \
     s|__MODEL_ID__|$MODEL_ID|g" services/llm/service.yaml >
services/llm/service.deployed.yaml
```

```
gcloud run services replace services/llm/service.deployed.yaml --region=$REGION
```

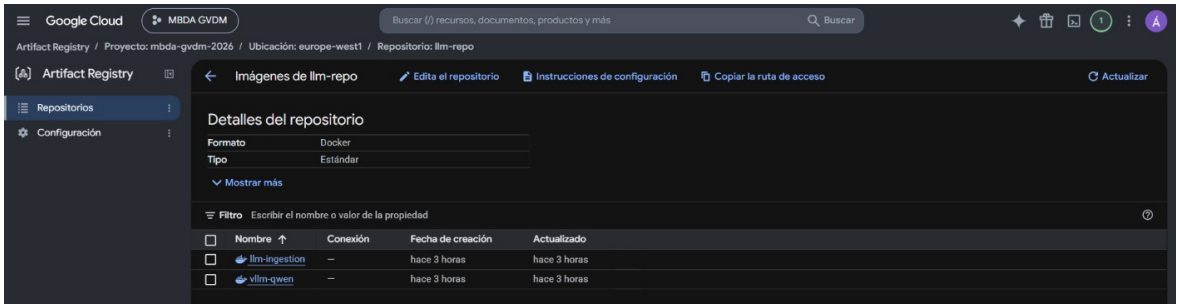


Figura 10. Vista interior de `llm-repo` en Artifact Registry con los dos contenedores: `llm-ingestion` (job) y `vllm-qwen` (servidor).

El manifiesto `service.yaml` es la pieza que define el despliegue con GPU: solicita una **NVIDIA L4**, 8 vCPU y 32 GiB de memoria, y monta el bucket en `/mnt/models` mediante el driver GCS FUSE nativo de Cloud Run, de modo que vLLM lee los pesos como si fueran un directorio local. El servicio se publica con `gcloud run services replace` y, para las pruebas del taller, se expone concediendo `roles/run.invoker` a `allUsers`; después responde en `/v1/chat/completions` con la API compatible con OpenAI.

El servicio con GPU L4 no se ha mantenido levantado: es el recurso más caro del despliegue y el propio enunciado exige eliminar toda la infraestructura al terminar. Las evidencias solicitadas se refieren al bucket, el repositorio, la cuenta de servicio, el job y la limpieza final.

Fase 7. Eliminación de todos los recursos cloud

Al finalizar se eliminan el *job*, el bucket con los pesos, el repositorio de imágenes y la cuenta de servicio, y se comprueba que no queda nada:

```
gcloud run jobs delete llm-ingestion-job --region=$REGION --quiet
gcloud storage rm --recursive gs://$BUCKET_NAME --quiet
gcloud artifacts repositories delete llm-repo --location=$REGION --quiet
gcloud iam service-accounts delete $SA_EMAIL --quiet
```

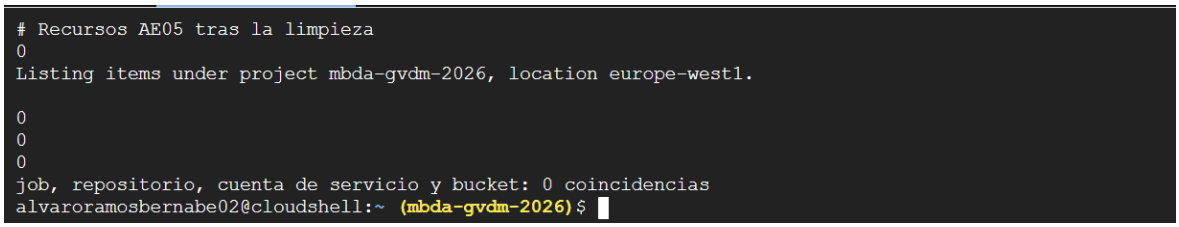


Figura 11. Comprobación tras la limpieza: 0 coincidencias para el job, el repositorio, la cuenta de servicio y el bucket.

Conclusiones

- El patrón **ingesta desacoplada + montaje FUSE** convierte el arranque del modelo en una operación de lectura local: el servidor no depende de Hugging Face en tiempo de arranque, lo que reduce el arranque en frío y elimina un punto de fallo externo.

- Cloud Run cubre las dos naturalezas de carga del MLOps de inferencia con la misma tecnología: *jobs* para tareas finitas (ingesta, reentrenamiento) y servicios con GPU para la inferencia en línea, ambos escalando a cero cuando no se usan.
- La identidad dedicada con permisos acotados al bucket es lo que hace que este esquema sea trasladable a producción: ni el job ni el servicio tienen permisos sobre el resto del proyecto.
- El control del coste forma parte del despliegue, no es un añadido: la GPU L4 se factura por tiempo de instancia activa, de modo que la limpieza de recursos es el último paso del procedimiento.